

Git & GitHub Notes

1. What is Git?

Git is a **version control system (VCS)** used to track changes in files, especially source code.

Think of Git as a **save-point system for your project**.

For example:

```
Project
├── Code
├── Documentation
└── Configurationbot Project
```

You make changes:

```
Version 1 → Version 2 → Version 3 → Version 4
```

Git allows you to:

- Track changes
- Go back to an older version
- Create different versions of a project
- Work with other developers
- See who changed what
- Merge changes from different developers

2. What is GitHub?

GitHub is an online platform that hosts Git repositories.

Simple difference:

Git	GitHub
Software/tool	Online platform
Works mainly on your computer	Works over the internet
Tracks project changes	Stores and shares Git repositories
Can work offline	Usually requires internet
Command-line tool	Website + collaboration platform

Easy example

```
Git
↓
Tracks your project locally

GitHub
↓
Stores your Git repository online
```

You can use Git **without** GitHub.

3. Git vs GitHub

Imagine you are working on a ROS2 project.

```
My ROS2 Robot
|
↓
Git
|
↓
Local computer
```

You can then upload it to:

```
GitHub
  ↓
Online repository
  ↓
Other developers can access it
```

4. What is a Repository?

A **repository (repo)** is a project folder that Git tracks.

Example:

```
my_robot/
├── src/
├── README.md
├── package.xml
├── CMakeLists.txt
└── .git/
```

The `.git` folder contains information Git uses to track the project.

When you run:

```
git init
```

Git creates the `.git` directory.

5. Installing Git

On Ubuntu:

```
sudo apt update
sudo apt install git -y
```

Check installation:

```
git --version
```

Example:

```
git version 2.43.0
```

6. Configure Git

Before using Git, configure your name and email.

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your@email.com"
```

Check configuration:

```
git config --list
```

7. Basic Git Workflow

The most important Git workflow is:



And when getting changes from GitHub:

```
GitHub
  ↓
git pull
  ↓
Local Repository
```

8. git init

Creates a new Git repository.

Example:

```
mkdir robot_project
cd robot_project
git init
```

To change default branch name from master to main:

```
git branch -m main
```

Output may look like:

```
Initialized empty Git repository
```

Now Git starts tracking this project.

To check whether git has initialised, check by using: `ls -la`

We can see

```
drwxrwxr-x 3 electrosage electrosage 4096 Aug 24 13:56 .
```

```
drwxrwxr-x 4 electrosage electrosage 4096 Aug 24 13:55 ..
```

```
drwxrwxr-x 7 electrosage electrosage 4096 Aug 24 13:56 .git  <-- This
confirms Git is initialized!
```

```
-rw-rw-r-- 1 electrosage electrosage 120 Aug 24 13:56 .gitignore
```

9. git status

Shows the current status of your repository.

```
git status
```

Example:

```
Untracked files:
  robot.cpp
  README.md
```

This tells you what Git knows about and what it doesn't.

10. Untracked Files

Suppose you create:

```
robot.cpp
```

Git initially sees:

```
robot.cpp → Untracked
```

Git is basically saying:

"I see this file, but you haven't told me to include it in the next version."

11. git add

Moves changes into the **staging area**.

For one file:

```
git add robot.cpp
```

For all changed files:

```
git add .
```

Now:

```
robot.cpp  
↓  
git add  
↓  
Staging Area
```

12. What is Staging?

The staging area lets you decide **which changes should be included in the next commit**.

Example:

Files:

```
robot.cpp ← modified  
sensor.cpp ← modified  
README.md ← modified
```

You can stage only:

```
git add robot.cpp
```

Then:

```
robot.cpp → staged  
sensor.cpp → not staged  
README.md → not staged
```

13. git commit

A **commit** is a saved version/snapshot of your project.

```
git commit -m "Add robot movement code"
```

Example:

```
git add .  
↓  
git commit -m "Add ultrasonic sensor"
```

A commit should have a meaningful message.

Good:

```
git commit -m "Add obstacle detection"
```

Bad:

```
git commit -m "changes"
```

14. Important Concept

```
git add  
↓  
Prepare changes  
  
git commit  
↓  
Save changes locally  
  
git push  
↓  
Upload changes to GitHub
```

Commit does NOT automatically upload your code to GitHub.

15. git log

Shows previous commits.

```
git log
```

Example:


```
commit abc123
```

```
Author: Rahul
```

```
Message: Add obstacle detection
```

```
commit def456
```

```
Author: Rahul
```

```
Message: Add motor control
```

```
commit ghi789
```

```
Author: Rahul
```

```
Message: Initial project
```

A shorter version:

```
git log --oneline
```

Example:

```
abc123 Add obstacle detection
```

```
def456 Add motor control
```

```
ghi789 Initial project
```

16. Git Branches

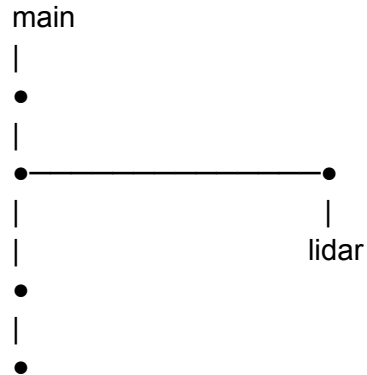
A **branch** is an independent line of development.

Imagine your main project:

main

|
•
|
•
|
•

You want to experiment with LIDAR:



The LIDAR work can happen separately without immediately changing `main`.

17. Why Use Branches?

Suppose your main robot software works perfectly.

You want to add:

`LIDAR`

Instead of directly modifying `main`:

`main`

↓

`Modify everything`

↓

`Robot breaks`

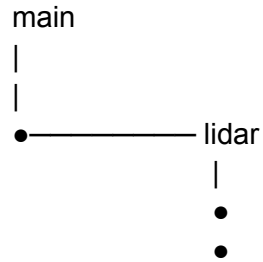
Create a branch:

`git branch lidar`

Switch to it:

`git switch lidar`

Now:



18. Create and Switch to a Branch

Modern Git:

```
git switch -c lidar
```

This means:

```
Create branch → lidar
+
Switch to it
```

Check branches:

```
git branch
```

Example:

```
* lidar
main
```

* means the branch you are currently using.

19. Switching Branches

```
git switch main
```

Go back to:

```
main
```

Then:

```
git switch lidar
```

Go back to:

```
lidar
```

20. Merging Branches

Suppose you completed your LIDAR feature.

You want to combine it with `main`.

First:

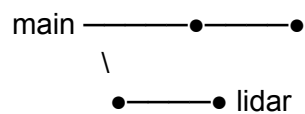
```
git switch main
```

Then:

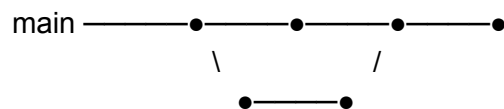
```
git merge lidar
```

Conceptually:

Before:



After:



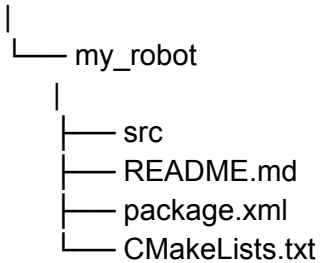
The changes from `lidar` are now incorporated into `main`.

21. What is GitHub Repository?

A GitHub repository is an online version of your project repository.

Example:

GitHub



22. Creating a GitHub Repository

On GitHub:

New Repository



Repository name:

my_robot



Create repository

You will get a remote repository.

A **remote** is a repository located somewhere else, usually GitHub.

23. Local vs Remote Repository

This is very important.

Local repository

Your computer:

Ubuntu

↓

~/my_robot

Remote repository

GitHub:

GitHub

↓

github.com/username/my_robot

Relationship:

LOCAL

REMOTE

my_robot ————— push —————→ GitHub

my_robot ←———— pull ————— GitHub

24. git remote

Check remote repositories:

```
git remote -v
```

Example:

origin https://github.com/user/my_robot.git (fetch)

origin https://github.com/user/my_robot.git (push)

origin is the conventional name given to the main remote repository.

25. Connecting Local Repository to GitHub

After creating a GitHub repository:

```
git remote add origin https://github.com/USERNAME/my_robot.git
```

Check:

```
git remote -v
```

26. git push

Uploads local commits to GitHub.

```
git push origin main
```

Meaning:

```
git push
  ↓
Send commits
  ↓
origin
  ↓
main branch
  ↓
GitHub
```

For the first push, you may use:

```
git push -u origin main
```

`-u` sets the upstream relationship, so later you can usually just use:

```
git push
```

27. git pull

Downloads changes from GitHub and integrates them into your current branch.

```
git pull
```

Conceptually:

```
GitHub  
↓  
Download changes  
↓  
Your local repository
```

Example:

Your teammate changes:

```
motor.cpp
```

They push it to GitHub.

You can get their changes:

```
git pull
```

28. git clone

`git clone` is used when you want to download an existing GitHub repository.

```
git clone https://github.com/USERNAME/my\_robot.git
```

- GitHub removed support for standard account passwords over HTTPS in 2021. You must use a Personal Access Token (PAT) or the GitHub CLI to authenticate.

Generate a Personal Access Token (Fastest)

1. Open **GitHub.com** in your browser and log in.
2. Go to **Settings** (click your profile picture top right → Settings).

3. Scroll down the left sidebar and click **Developer settings** → **Personal access tokens** → **Tokens (classic)**.
4. Click **Generate new token** → **Generate new token (classic)**.
5. Give it a note (e.g., **Laptop Terminal**), check the top **repo** scope box, scroll down, and click **Generate token**.
6. **Copy the token** (it starts with **ghp_...**).

Now rerun **git clone**:

Bash (example)

```
git clone https://github.com/RohithPVenugopal/Python.git
```

- **Username:** **RohithPVenugopal**
- **Password:** *Paste your generated token here* (the text won't show while pasting—that's normal, just hit Enter).

This creates:

```
my_robot/
```

on your computer.

Difference

```
git init
```

Creates a **new Git repository**.

```
git clone
```

Copies an **existing repository**.

29. GitHub Workflow

A common workflow is:

```
git clone <repository>
cd my_robot
```

Make changes:

Edit code

Check:

```
git status
```

Stage:

```
git add .
```

Commit:

```
git commit -m "Add motor controller"
```

Upload:

```
git push
```

30. Complete Example

Suppose you create a robot project.

Step 1 — Create folder

```
mkdir robot_project  
cd robot_project
```

Step 2 — Initialize Git

```
git init
```

Step 3 — Create code

```
nano robot.cpp
```

Step 4 — Check status

```
git status
```

Step 5 — Stage

```
git add robot.cpp
```

Step 6 — Commit

```
git commit -m "Add initial robot program"
```

Step 7 — Connect GitHub

```
git remote add origin https://github.com/USERNAME/robot_project.git
```

Step 8 — Push

```
git branch -M main  
git push -u origin main
```

Now the project is available on GitHub.

31. .gitignore

.gitignore tells Git which files/folders **not to track**.

Example:

```
build/  
install/  
log/  
*.pyc  
.env
```

For ROS2 projects, you commonly don't want generated folders such as:

```
build/  
install/  
log/
```

Example **.gitignore**:

```
build/  
install/  
log/
```

Then:

```
git add .
```

Git will ignore those folders.

32. Why **.gitignore** is Important

Suppose your ROS2 workspace contains:

```
robot_ws/  
├── src/  
├── build/  
├── install/  
└── log/
```

Usually you want GitHub to contain:

```
robot_ws/  
└── src/
```

and not:

```
build/  
install/  
log/
```

because these are generated files.

33. Git Diff

`git diff` shows changes that haven't been staged.

```
git diff
```

Example:

```
- int speed = 50;  
+ int speed = 80;
```

It tells you exactly what changed.

34. `git diff --staged`

Shows changes that have already been staged.

```
git diff --staged
```

Useful before committing.

35. Undoing Changes

If you modified a file but haven't staged it:

```
git restore robot.cpp
```

This restores the file to its previous Git version.

 Be careful: your uncommitted changes can be lost.

36. Unstage a File

Suppose:

```
git add robot.cpp
```

but you don't want it staged anymore.

Use:

```
git restore --staged robot.cpp
```

The changes remain in the file, but they are removed from staging.

37. Git Merge Conflict

A conflict happens when two people modify the same part of a file differently.

Example:

```
Developer A:  
speed = 50;
```

```
Developer B:  
speed = 80;
```

Git may not know which one is correct.

You may see:

```
<<<<<< HEAD  
speed = 50;  
=====  
speed = 80;  
>>>>>> branch
```

You manually decide what the final code should be.

Then:

```
git add .  
git commit -m "Resolve merge conflict"
```

38. GitHub Pull Request

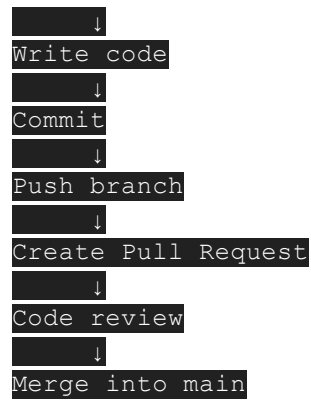
A **Pull Request (PR)** is a request to merge changes from one branch into another.

Example:

```
feature-lidar  
|  
| Pull Request  
↓  
main
```

Typical workflow:

```
Create branch
```



39. Fork

A **fork** is your own copy of someone else's GitHub repository under your GitHub account.

Example:



Forks are commonly used when contributing to open-source projects.

40. Fetch vs Pull

git fetch

Downloads information about changes from the remote repository but doesn't automatically merge them.

```
git fetch
```

git pull

Fetches changes **and integrates them** into your current branch.

```
git pull
```

Simple:

```
fetch = "Tell me what's new."
```

```
pull = "Get the new changes and integrate them."
```

41. Important Git Commands

Command	Purpose
<code>git init</code>	Create Git repository
<code>git status</code>	Check repository status
<code>git add .</code>	Stage all changes
<code>git add file.cpp</code>	Stage one file
<code>git commit -m "message"</code>	Save changes
<code>git log</code>	View commit history
<code>git clone</code>	Download repository
<code>git push</code>	Upload commits
<code>git pull</code>	Get remote changes
<code>git fetch</code>	Download remote information

<code>git branch</code>	View branches
<code>git switch</code>	Change branch
<code>git merge</code>	Merge branches
<code>git remote -v</code>	View remote repository
<code>git diff</code>	View changes
<code>git restore</code>	Restore changes

42. The Most Important Commands to Remember

For beginners, remember this sequence:

```
git status  
  
git add .  
  
git commit -m "Describe your changes"  
  
git push
```

Before starting work on an existing project:

```
git pull
```

So the basic daily workflow is:

```
START  
↓  
git pull  
↓  
Write/Edit Code  
↓  
git status  
↓
```

```
git add .  
↓  
git commit  
↓  
git push  
↓  
DONE
```

43. Git Terminology

Repository

A project tracked by Git.

Commit

A saved snapshot/version of your project.

Branch

A separate line of development.

Remote

A repository located somewhere else, such as GitHub.

Origin

The conventional name for the main remote repository.

Push

Upload commits to a remote repository.

Pull

Download and integrate remote changes.

Clone

Copy an existing repository.

Merge

Combine changes from different branches.

Staging Area

A temporary area where you select changes before committing.

Pull Request

A request to merge changes into another branch on GitHub.

44. Simple Real-World Analogy

Think about writing a robotics project report.

Your notebook



Git

Every time you finish an important section:

Version 1

Version 2

Version 3

Version 4

Git remembers those versions.

GitHub is like putting that notebook into an **online shared workspace**:

Your Computer



Git



GitHub



Other developers

So remember:

Git = Version control tool

GitHub = Online platform for Git repositories

One-line memory trick

Git → Track

GitHub → Share

Commit → Save

Push → Upload

Pull → Download

Clone → Copy

Branch → Separate work

Merge → Combine